How the Fraud Triage Prototype Works

Junior AI Investigator - the process, step by step, in plain English

In one sentence: every morning, a fixed set of rules scores each insurance claim for fraud risk, an AI writes a short plain-English explanation of each case and cites the facts it used, a fact-checker verifies those citations, and an investigator reviews the queue in a web app where they can accept or override the AI, ask it questions, and leave a permanent record of every decision.

What	Details
Data	50 synthetic home-care claims (data/sample_cases.csv)
Rules	10 weighted warning signs, score 0-100, Low / Medium / High
AI	Groq (free tier): gpt-oss-20b and gpt-oss-120b, run in parallel
Checker	Verifies every fact the AI cites against the real data
App	FastAPI backend + a plain HTML/JS page; SQLite for decisions
Side-by-side scores	The automatic check's score and the AI's own score, both out of 100
Full run time	About 2 minutes for all 50 cases (115 seconds)
Tests	45 automated tests, all passing

Contents

1. The problem we are solving
2. The whole process at a glance (flowchart)
3. Step 1 - Load the cases
4. Step 2 - The automatic check (rules)
5. Step 3 - Compare each case with the others
6. Step 4 - The AI assessment
7. Step 5 - The fact-checker
8. Step 6 - Saving the results
9. Step 7 - The investigator app
10. How well it works (results)
11. Key design choices and why
12. Problems I faced and how I solved them
13. Bottlenecks today
14. What I can do next
15. Alternatives (and which are most effective)
16. How to run it
17. Where everything lives (file map)
18. Glossary

1. The problem we are solving

An insurance fraud team gets a pile of flagged claims every morning. Most are innocent; a few are real fraud. An investigator has limited time, so the job is **triage**: decide which cases to look at first, what to do with each one, and close the obvious false alarms quickly.

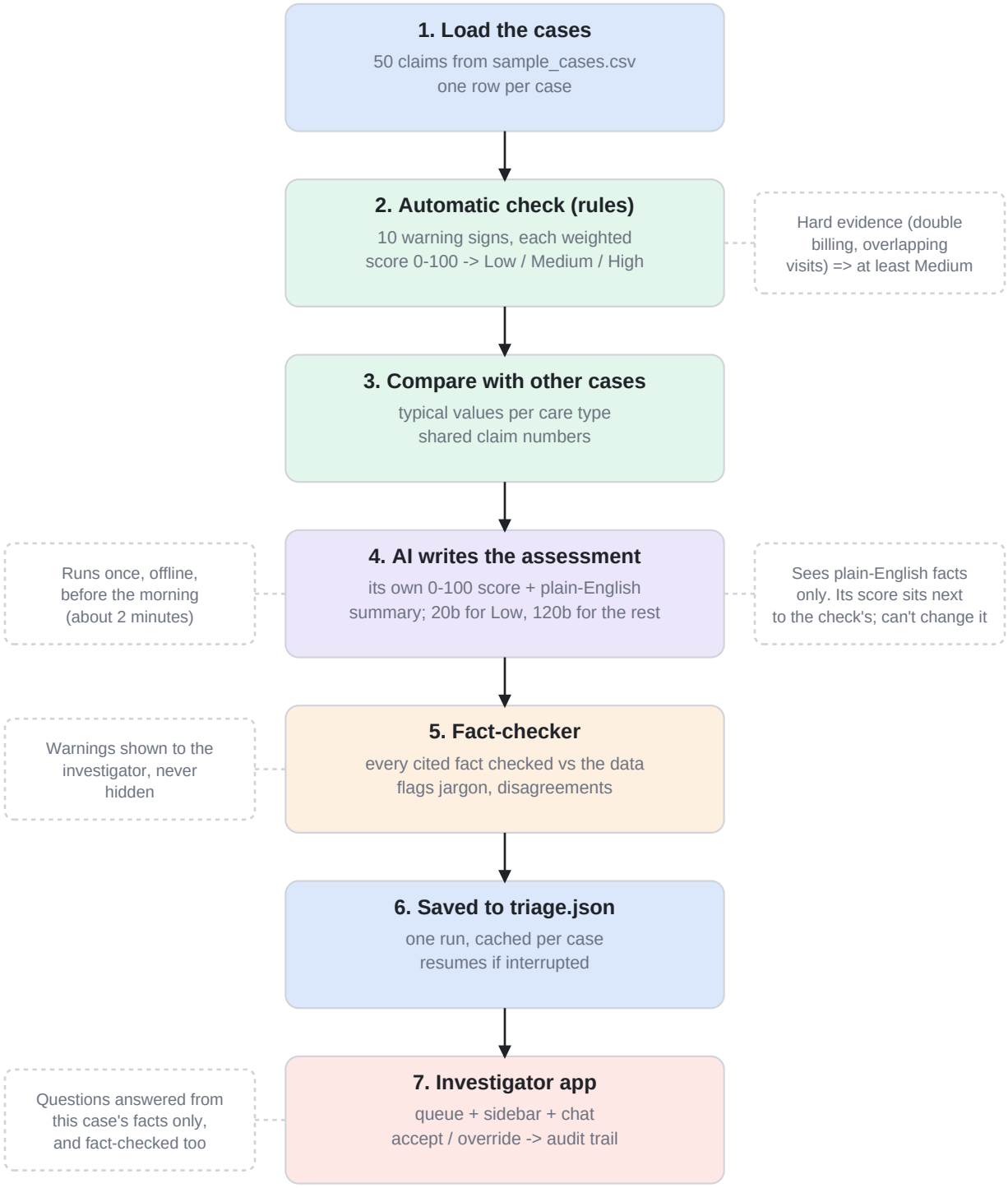
The goal of this prototype is a **junior AI investigator** that does the first pass for them: sort the queue by risk, explain each case in words anyone can follow, and suggest one of three actions:

- **Escalate to SIU** - send to the Special Investigations Unit (serious fraud team).
- **Request documentation** - ask the provider for records before deciding.
- **Close as false positive** - nothing suspicious; close the alert.

The human investigator always makes the final call. The AI's job is to save them time and to make its reasoning easy to check - never to decide on its own.

2. The whole process at a glance

Steps 1-6 run once, ahead of time, from a single command (`python scripts/run_triage.py`). Step 7 is the web app the investigator uses during the day.



3. Step 1 - Load the cases

The input is a spreadsheet (CSV) of 50 synthetic claims. Each row is one case with an ID (C1001 to C1050), a claim number, the type of care, the amount billed, and a set of measurements such as visits per week, distance between patient and provider, how often bills are round numbers, and yes/no flags such as "the same service was billed twice".

Column names like `weekly_visit_frequency` mean nothing to a non-expert, so a small dictionary (`labels.py`) translates every column into a plain label ("Visits per week") and formats every value ("Yes", "$1,250", "44% of bills", "85 miles"). This translation is used everywhere a person sees data.

4. Step 2 - The automatic check (rules)

Before any AI is involved, a deterministic scorer looks at ten **warning signs**. Each one has a weight that reflects how strongly it points at fraud:

Warning sign	Weight	Triggers when
Same service billed twice	3.0	Yes
Another provider billed for the same time	3.0	Yes
Patient and provider share contact details	2.5	Yes
Amount compared with similar claims	2.0	25% above (half) / 75% above (full)
Visits per week	2.0	7+ (half) / 12+ (full)
Distance between patient and provider	2.0	40+ miles (half) / 100+ (full)
Bills in round dollar amounts	1.5	30% of bills (half) / 60% (full)
Billing on weekends	1.5	20% of bills (half) / 40% (full)
Policy changed just before the claim	1.5	Yes
Claims in the past year	1.0	4+ (half) / 8+ (full)

How the score is built. Each sign earns nothing, half its weight ("elevated") or its full weight ("severe"). The points are added up and turned into a score out of 100. Then:

- **50 or more** -> High risk
- **20 to 49** -> Medium risk
- **below 20** -> Low risk

The safety floor. Some evidence is too strong to ever auto-close. If the same service was billed twice, another provider billed for the same time, or the case shares its claim number with another case, the case is raised to **at least Medium** no matter what the score says, and the reason is recorded (for example "the same service was billed twice").

Why rules first? Rules are fast, free, repeatable and explainable: anyone can open `config.py` and see exactly why a case scored what it did. The rules decide the risk level; the AI explains it.

5. Step 3 - Compare each case with the others

A number on its own is hard to judge. Is 9 visits a week a lot? It depends on the type of care. So for each care type the pipeline works out the **typical** (median) amount, weekly visits and distance, and expresses each case as a multiple: "visits are **2.1 times the usual** for this type of care".

We use "times the usual" rather than percentages because in testing the AI misread percentages ("115% of" vs "115% above"). Multiples are precomputed so the AI never has to do arithmetic.

It also spots cases that **share a claim number** - the same claim showing up twice - which is a strong sign of duplicate billing.

These comparisons feed the **"Main concerns"** column in the app: shared claim numbers first, then triggered warning signs, then any value that is at least 2 times the usual and wasn't already counted.

6. Step 4 - The AI assessment

What the AI receives. For each case the AI is given a compact bundle of facts in plain English: the case's own data, the automatic check's result (score, risk level, which warning signs fired, whether the safety floor applied and why), and the comparisons with similar cases. It is not given the internal column jargon or the rules' code.

What it must return. A strict JSON answer with:

- its **own risk score out of 100**, judged from the evidence rather than copied from the automatic check, on the same scale (under 20 Low, 20-49 Medium, 50 or more High),
- a **risk level** (Low / Medium / High) that matches that score, and a **confidence**,
- a **recommended action** (one of the three actions),
- a **short summary** (about 55 words on average) written for a non-expert,
- a list of **key facts** - each one names the exact data field it relies on, its value, and why it matters.

Plain-English rules in the prompt. The instructions tell the AI to write for someone with no knowledge of the data: no column names, no "tiers", "signals", "medians" or "ratios" - say "the usual for this type of care" instead.

Which model. We use Groq's free tier with two open models:

Cases	Model	Thinking effort	Why
Low risk (30)	gpt-oss-20b (smaller, faster)	low	Simple cases; speed matters
Medium (9)	gpt-oss-120b (larger)	medium	Borderline: most judgement needed
High (11)	gpt-oss-120b	low	Evidence is already strong

Two lanes in parallel. Groq's free tier limits each model separately (about 8,000 tokens per minute and 200,000 per day). So the two models run at the same time, each working through its own list. This cut a full run from roughly 6-7 minutes to about **2 minutes** (115 seconds in the latest run).

Two API keys. You can put several Groq keys in `.env`, separated by commas. Each request tries the first key; if that key is rate-limited it moves to the next; if a key has used its daily allowance for a model it is skipped for the rest of the day; a broken key is skipped entirely. If every key is busy, the lane waits and retries.

Resumable and cached. Each finished case is saved immediately. If the run stops halfway (network error, daily limit), running it again only does the missing cases. A "prompt version" number is stored with each result, so changing the instructions automatically triggers a fresh assessment.

7. Step 5 - The fact-checker (grounding)

AI models can make things up. So nothing the AI says is shown as fact until it is checked. For every key fact the AI cites, the checker looks up that field in the real data and compares the value. Matches get a green tick in the app; mismatches get an amber warning.

The checker also raises plain-English warnings when:

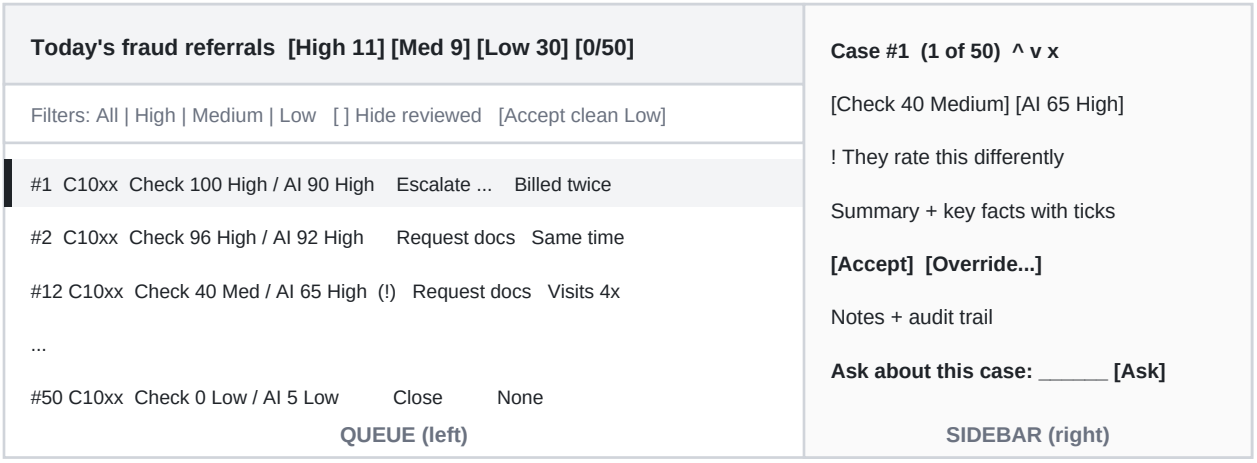
- **The AI disagrees with the automatic check** - "The AI rates this High risk (65 out of 100), but the automatic check rates it Medium (40 out of 100). Worth a closer look."
- **The AI's score doesn't match its own level** - e.g. 35 out of 100 but called High.
- **The AI suggests closing a case that has hard evidence** (double billing, overlapping visits) - this should never happen silently.
- **Technical wording leaks through** (words like "ratio" or column-style names).

Warnings are always shown to the investigator, never hidden. The same checker also runs on every answer in the chat.

8. Step 6 - Saving the results

Everything is written to `outputs/triage.json`: for each case the score, risk level, warning signs, floor reason, the AI assessment, the checker's warnings, which model was used and the prompt version. The web app only *reads* this file - it never re-scores or re-runs the AI on page load, so the app is instant and costs nothing to browse.

9. Step 7 - The investigator app



The queue (left). All 50 cases, highest risk first, each with a serial number, both risk scores (the check's above the AI's, with "Levels differ" when they disagree), review status, the AI's suggested action and summary, and the main concerns. Rows are kept to two lines so about eight fit on screen. Above the queue, four summary cards show the High, Medium and Low counts (click one to filter) and a progress bar. Filters and a "Hide reviewed" switch stay pinned at the top while scrolling.

The sidebar (right). On a desktop the first case opens automatically. At the top, two cards side by side compare the **automatic check** (score, level, number of warning signs) with the **AI** (score, level, confidence), each with a coloured bar, and a line saying whether they agree. Below that come the suggested action, the summary, each key fact with its tick or warning, the full claim data in plain words, and any checker warnings. The up/down buttons or arrow keys move between cases, with a "3 of 50" position.

Decisions. The investigator types their name once (remembered by the browser), then:

- **Accept** - agree with the AI's risk level and action.
- **Override** - choose a different risk level and/or action and give a reason. The server rejects an "override" that is identical to the AI's call, so the record stays honest.
- After a decision, the next case still to review opens automatically.

Bulk accept clean Low cases. One button accepts every case where the rules and the AI both say Low, the AI says close, there is no hard evidence and no checker warning. The server re-checks each case itself, so an out-of-date page can never bulk-close a case that needs a look.

Notes and audit trail. Every decision and note is stored in a small database (`outputs/decisions.db`) with who, when, what the AI said and what the human decided. Records are only ever added, never edited or deleted, so there is a full history of each case.

Ask about this case. A chat box at the bottom of the sidebar lets the investigator ask questions like "Why is the distance a concern?". The AI answers using only this case's facts, remembers the last few questions, says so when the data can't answer, and its citations go through the same

fact-checker.

Phones. On a small screen the queue fills the page and a case opens full-screen when tapped.

10. How well it works (results)

There are no "true fraud" labels in the synthetic data, so we cannot measure accuracy directly. Instead `scripts/evaluate.py` measures what can be measured. Latest full run:

Measure	Result
Cases assessed by the AI	50 of 50 (30 on 20b, 20 on 120b)
Same risk level as the automatic check	43 of 50 (86%); Cohen's kappa about 0.75 (substantial)
Where they differ	7 of the 9 Medium cases the AI rated High (more cautious, never less)
AI score vs check score	10.5 points apart on average; AI higher on 40 of 50
AI score matches its own level	50 of 50
Cited facts confirmed by the checker	416 of 416 (100%)
Cases with any warning	12 of 50 (7 for the word "ratio", 7 disagreements)
Average summary length	55 words
Full run time	115 seconds (two keys, two lanes)
Override rate	Measured once investigators use the app

Agreement table (rows = automatic check, columns = AI):

	AI: Low	AI: Medium	AI: High
Rules: Low	30	0	0
Rules: Medium	0	2	7
Rules: High	0	0	11

Average scores by risk level (automatic check vs AI):

Risk level (check)	Cases	Check score	AI score
High	11	85	82
Medium	9	25	57
Low	30	0	5

What this shows: the two agree closely on High and Low cases. On the borderline Medium cases the AI is much more cautious (57 vs 25 on average), weighing duplicate billing and shared contact details more heavily. These are exactly the cases worth an investigator's attention, and it may also mean the check's Medium band is a little lenient.

Suggested actions: every Low case -> close (30); Medium -> 8 request documentation, 1 escalate; High -> 7 escalate, 4 request documentation.

11. Key design choices and why

Choice	Why
Rules set the risk level; the AI explains it	Repeatable, auditable and cheap. The AI can disagree, but that is flagged rather than silently applied.
AI gives its own score, shown side by side	Two independent opinions on the same 0-100 scale; a disagreement is a signal to look closer. The check stays the official ranking.
Hard-evidence safety floor	A double-billed case must never be auto-closed, whatever the score.
Every AI fact is checked	Investigators can trust the ticks and see exactly where the AI might be wrong.
Plain English everywhere	The person reading may not know the data or fraud jargon.
Precomputed comparisons	The AI makes arithmetic mistakes; giving it "2.1 times the usual" removes them.
Run once, cache results	The app is instant and free to browse; reruns only redo what changed.
Two models, two lanes, two keys	Fits the free tier: each model and key has its own limit, so they work side by side.
Human makes the final call; append-only log	Accountability: who decided what, when, and why they disagreed with the AI.

12. Problems I faced and how I solved them

Building this was not a straight line. These are the real problems hit along the way, grouped by area, with the fix for each.

Speed and cost

Problem	Why it happened	Fix
A full run took 6-7 minutes for 50 cases	One large model doing every case in a row, with long prompts and full JSON schemas	Small model for Low cases, large for the rest, both running at once; compact facts; short output descriptions; low thinking effort. Now about 2 minutes
Hit Groq's daily limit (200k tokens) mid-run	A full refresh on the large model used up the day's allowance	Fall back to the other model, then support several keys: each key is tried in turn and skipped for the day once used up
Rerunning after a crash redid everything	Results were only saved at the end	Save each case as soon as it finishes; a rerun only does missing cases
Earlier versions tried Gemini and Claude too	Multiple providers added code and setup with no gain on the free budget	Kept Groq only; removed the other libraries

AI quality

Problem	Why it happened	Fix
Text full of jargon ("tier", "signal", "median", column names)	The prompt's own wording and the internal field names primed the model to repeat them	Renamed every fact sent to the AI into plain words; removed the jargon from the prompt itself; checker flags any that slips through
14 cases cited labels instead of field names, so citations could not be checked	The "never write field names" rule confused the model about the citation format	Added an exact example of the citation format to the prompt
AI misread percentages ("115% of" vs "115% above")	Percentages are ambiguous, and models are weak at arithmetic	Precomputed "X times the usual" so the AI never calculates
C1002: two different "usual" amounts disagreed	The data had its own comparison column and we computed another	Gave each a distinct plain name and precomputed both
C1001 was missed (rated Low)	It shares its claim number with another case, which no single row shows	Cross-case check for shared claim numbers plus a safety floor to at least Medium

Problem	Why it happened	Fix
C1033: AI suggested closing a High-risk case	Low thinking effort on a case with conflicting signs	The checker flags it to the investigator (an automatic re-ask is a planned next step)
"Main concerns" empty for most cases after the model split	The column came from the AI's text, which the small model kept brief	Build concerns from the rules and comparisons instead, which is consistent for every case

Engineering and app

Problem	Why it happened	Fix
A missing key failed every case one by one	The config error was caught as an ordinary AI error	Treat it separately: stop both lanes at once with a clear message
Browser kept showing the old page after changes	Static files were cached	Server tells browsers to recheck page files each time
Phone layout scrolled sideways by 8px	Mobile styles were overridden by rules later in the file	Moved the mobile rules to the end of the stylesheet
Tests wrote into the real decisions database	The tests used the real file path	Tests use a temporary database
An API key was printed in the terminal	Running `source .env` with a space after a comma	Never source .env; keys are loaded in Python. Rotate any exposed key

13. Bottlenecks today

Bottleneck	Impact	How bad
Free-tier rate limits (about 8,000 tokens/minute and 200,000/day per model)	Caps how fast and how many cases can be assessed; a few hundred cases a day at most	Biggest limit on scale
AI response time (a few seconds per case)	Over 90% of run time is waiting for the model	High
Each lane does one case at a time	Can't use more parallelism because of the per-minute limit	Medium
No fraud labels	Can't measure true accuracy, only agreement and grounding	High for trust
Hand-set rule weights and thresholds	May miss patterns the rules don't describe	Medium
Chat calls the AI live	Each question takes a few seconds and uses quota	Low
triage.json re-read on each request; SQLite for decisions	Fine for one team, not for many users at once	Low now, high at scale
Typed names instead of logins	Audit trail can't prove who decided	Must fix before real use

14. What I can do next

Quick wins (hours)

- **Review the Medium band:** the AI rated 7 of the 9 Medium cases High; check with investigators whether the check's thresholds are too lenient.
- **Auto re-ask on contradictions:** when the AI suggests closing a case with hard evidence, or disagrees with the rules, ask again with more thinking before saving.
- **Remove the last jargon** ("ratio") by adding it to the prompt's plain-wording list and the fact names.
- **Stream chat answers** word by word so they feel instant.
- **Schedule the run** nightly (cron or Airflow) so the queue is ready each morning.

Medium effort (days)

- **A golden test set:** 20-30 cases with agreed correct answers, run on every prompt change to catch regressions.
- **Learn from overrides:** track which warning signs investigators disagree with and retune weights and thresholds.
- **Proper sign-in and roles** so the audit trail is trustworthy.
- **Explain with visuals:** a small chart per case showing each value against the usual.
- **Postgres instead of SQLite** and a database instead of triage.json for multi-user use.

Bigger steps (weeks, needs real data)

- **Machine-learning risk score** trained on labelled outcomes, kept alongside the rules for explainability.
- **Network analysis** to find rings: providers and patients linked by shared phones, addresses or claim numbers.
- **Monitoring:** track agreement, grounding rate and override rate over time and alert when they drift.

15. Alternatives (and which are most effective)

The current choices were driven by a zero budget. Here are the main alternatives for each part, with a recommendation. **Bold** marks the most effective option for that need.

The AI model

Option	Strengths	Weaknesses	Best for
Groq free tier (current)	Free, very fast, good open models	Strict daily and per-minute limits	Prototypes and demos
Local model (Ollama / vLLM with Llama, Qwen, gpt-oss)	Free, no limits, data never leaves the machine (important for health and claims data)	Needs a decent GPU/Mac; slower on a laptop	**Zero-cost and private; best free option at scale**
Gemini Flash free tier	Generous free quota, large context	Free tier may use data for training	A second free fallback
Groq / OpenAI paid	Same code, higher limits	Costs money	Easy upgrade path
Claude (e.g. Haiku 5.5 for volume, Sonnet 5.5 for hard cases) with Batch API	Strong instruction-following and structured output; batch processing is cheaper for overnight runs	Paid	**Production quality for a nightly run**

The risk score

Option	Strengths	Weaknesses	Best for
Weighted rules (current)	Transparent, no data needed	Hand-tuned, misses new patterns	Starting out, regulators
Anomaly detection (Isolation Forest)	No labels needed; finds unusual cases	Unusual is not the same as fraud; harder to explain	Finding new patterns
Gradient boosting (XGBoost / LightGBM) + SHAP explanations	Most accurate on tabular data; SHAP says which features drove the score	Needs labelled outcomes	**Industry standard once labels exist**
Graph analysis (NetworkX / Neo4j)	Finds collusion rings across cases	More setup	Organised fraud

Getting reliable structured output

Option	Strengths	Weaknesses	Best for
JSON mode + checker (current)	Works on free models; checker catches bad facts	Occasional retries	Now
Strict schema / tool calling (or the Instructor library)	The model cannot return the wrong shape	Not supported by every model	**Fewer retries, simpler code**
Rules write the text, no AI	Free, instant, never wrong	Robotic, can't answer questions	A fallback when the AI is down

App, storage and scheduling

Part	Current	Alternative	When to switch
Front end	Plain HTML/JS	Streamlit (faster to build) or React (richer)	Streamlit for a quick demo; React for a real product
Storage	JSON file + SQLite	**Postgres**	More than one investigator at a time
Scheduling	Run by hand	**cron or Airflow**	As soon as it runs daily
Hosting	Laptop	Docker on Render, Fly.io or a cloud VM	When others need access

My recommendation: for zero cost, keep the rules, move the AI to a **local model through Ollama** (no limits, and claims data stays private), and add the auto re-ask and a golden test set. Once real outcomes exist, add a **gradient-boosting model with SHAP** next to the rules, and keep the AI for what it does best: explaining cases in plain English and answering questions.

16. How to run it

1) Put your Groq key(s) in `.env` as `GROQ_API_KEY=key1,key2` (never commit this file).
2) Install: `pip install -r requirements.txt`
3) Score and assess all cases: `python scripts/run_triage.py` (add `--refresh` to redo everything)
4) Start the app: `uvicorn app.main:app --reload` and open http://localhost:8000
5) See the numbers: `python scripts/evaluate.py`
6) Run the tests: `pytest`

17. Where everything lives (file map)

File	What it does
src/triage/config.py	The 10 warning signs, weights, thresholds, safety floor
src/triage/scoring.py	Turns a case into a score, risk level and floor reason
src/triage/cross_case.py	Typical values per care type; shared claim numbers
src/triage/pipeline.py	Combines scoring + comparisons; builds "Main concerns"
src/triage/labels.py	Plain-English names and formatting for every field
src/triage/llm.py	Prompts, Groq calls, model choice, multi-key fallback, chat
src/triage/grounding.py	The fact-checker and plain-English warnings
src/triage/store.py	SQLite audit trail: decisions and notes
scripts/run_triage.py	The one command that runs steps 1-6 in two lanes
scripts/evaluate.py	Prints the results numbers
app/main.py	The web API: queue, case, ask, decision, notes, bulk accept
app/static/	The page: index.html, app.js, styles.css
tests/	45 automated tests

18. Glossary

Term	Meaning
Triage	Sorting cases so the most urgent get looked at first
SIU	Special Investigations Unit - the team that handles likely fraud
False positive	A case that was flagged but turns out to be innocent
Warning sign	One measurable thing that makes fraud more likely
Safety floor	Rule that keeps hard-evidence cases at Medium or above
LLM	Large language model - the AI that writes the text
Groq	The service that runs the open AI models quickly (free tier used)
Grounding	Checking that what the AI says is backed by the actual data
Token	A piece of a word; AI usage limits are counted in tokens
Audit trail	A permanent, add-only record of who decided what and when
Risk score	A number from 0 to 100; the check and the AI each give one, same bands

Term	Meaning
Cohen's kappa	Agreement between two raters after removing what chance alone would give